



## AZ-204T00A Learning Path 05: Implement containerized solutions

© Copyright Microsoft Corporation. All rights reserved.



These Learn modules are part of the AZ-204: [Implement containerized solutions](https://learn.microsoft.com/training/paths/az-204-implement-iaas-solutions/) (<https://learn.microsoft.com/training/paths/az-204-implement-iaas-solutions/>) learning path.

## Module Online Material

<https://learn.microsoft.com/en-us/training/paths/az-204-implement-iaas-solutions/>

### Implement containerized solutions

1 hr 42 min • Learning Path • 0 of 3 modules completed

#### At a glance

 Level

Intermediate

 Product

Azure, Azure Container Instances, Azure  
Container Registry

 Role

Developer

 Subject

Containers

Learn how to create and deploy containerized solutions to Azure by using the Azure Container Registry, Azure Container Instances, and Azure Container Apps.

<https://learn.microsoft.com/en-us/training/paths/az-204-implement-iaas-solutions/>

## Agenda

---

- Manage container images in Azure Container Registry
- Run container images in Azure Container Instances
- Implement Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

# Module 1: Manage Container Images in Azure Container Registry

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/training/modules/publish-container-image-to-azure-container-registry/>

## Learning objectives

- Explain the features and benefits Azure Container Registry offers
- Describe how to use ACR Tasks to automate builds and deployments
- Explain the elements in a Dockerfile
- Build and run an image in the ACR by using Azure CLI

## Azure Container Registry

- Azure Container Registry (ACR) is a managed registry service based on the open-source Docker Registry 2.0.
  - Organization can create and maintain Azure container registries to store and manage their container images and related artifacts.
- Developers can also push container images to a container registry as part of a container development workflow.
  - For example, target a container registry from a continuous integration and delivery tool such as GitHub, Azure Pipelines, etc.

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/en-us/training/modules/publish-container-image-to-azure-container-registry/2-azure-container-registry-overview>

## Discover the Azure Container Registry

Use the Azure Container Registry (ACR) service with your existing container development and deployment pipelines or use Azure Container Registry Tasks to build container images in Azure.

### Use cases

Pull images from an Azure container registry to various deployment targets:

- *Scalable orchestration systems* that manage containerized applications across clusters of hosts.
- *Azure services* that support building and running applications at scale.

### Azure Container Registry service tiers

Azure Container Registry is available in multiple service tiers.

- Basic
- Standard
- Premium

© Copyright Microsoft Corporation. All rights reserved.

### Use cases

**Scalable orchestration systems:** including Kubernetes, DC/OS, and Docker Swarm.

**Azure services:** including Azure Kubernetes Service (AKS), App Service, Batch, Service Fabric, and others.

Developers can also push to a container registry as part of a container development workflow. For example, target a container registry from a continuous integration and delivery tool such as Azure Pipelines or Jenkins.

Configure ACR Tasks to automatically rebuild application images when their base images are updated, or automate image builds when your team commits code to a Git repository. Create multi-step tasks to automate building, testing, and patching multiple container images in parallel in the cloud.

### Azure Container Registry service tiers

Azure Container Registry is available in multiple service tiers. These tiers provide predictable pricing and several options for aligning to the capacity and usage patterns of your private Docker registry in Azure.

**Basic** - A cost-optimized entry point for developers learning about Azure Container Registry. Basic registries have the same programmatic capabilities as Standard and Premium (such as Azure Active Directory authentication integration, image deletion, and webhooks). However, the included storage and image throughput are most

appropriate for lower usage scenarios.

**Standard** - Standard registries offer the same capabilities as Basic, with increased included storage and image throughput. Standard registries should satisfy the needs of most production scenarios.

**Premium** - Premium registries provide the highest amount of included storage and concurrent operations, enabling high-volume scenarios. In addition to higher image throughput, Premium adds features such as geo-replication for managing a single registry across multiple regions, content trust for image tag signing, private link with private endpoints to restrict access to the registry.



## ACR – Service Tiers

| Tier     | Description                                                                                                                                                                                                                                                                                                                                                                                  |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Basic    | A cost-optimized entry point for developers learning about Azure Container Registry. Basic registries have the same programmatic capabilities as Standard and Premium (such as Microsoft Entra authentication integration, image deletion, and webhooks). However, the included storage and image throughput are most appropriate for lower usage scenarios.                                 |
| Standard | Standard registries offer the same capabilities as Basic, with increased included storage and image throughput. Standard registries should satisfy the needs of most production scenarios.                                                                                                                                                                                                   |
| Premium  | Premium registries provide the highest amount of included storage and concurrent operations, enabling high-volume scenarios. In addition to higher image throughput, Premium adds features such as: geo-replication for managing a single registry across multiple regions, content trust for image tag signing, and private link with private endpoints to restrict access to the registry. |

© Copyright Microsoft Corporation. All rights reserved.

## Discover the Azure Container Registry ( 2 of 2 )

### Supported images and artifacts

- Grouped in a repository, each image is a read-only snapshot of a Docker-compatible container.
- Azure container registries can include both Windows and Linux images.
- Azure Container Registry also stores Helm charts and images built to the Open Container Initiative (OCI) Image Format Specification.

### Azure Container Registry Tasks

- Use Azure Container Registry Tasks (ACR Tasks) to streamline building, testing, pushing, and deploying images in Azure.

© Copyright Microsoft Corporation. All rights reserved.

**ACR Tasks:** Configure build tasks to automate your container OS and framework patching pipeline and build images automatically when your team commits code to source control.

## Explore storage capabilities

Every Basic, Standard, and Premium Azure container registry benefits from advanced Azure storage features.

- Encryption-at-rest: All container images in your registry are encrypted at rest.
- Geo-redundant storage: Azure uses a geo-redundant storage scheme to guard against loss of your container images.
- Geo-replication: For scenarios requiring even more high-availability assurance, consider using the geo-replication feature of Premium registries.
- Zone redundancy: Premium service tier, uses Azure availability zones to replicate your registry to a minimum of three separate zones in each enabled region.
- Scalable storage: Create as many repositories, images, layers, or tags as you need, up to the registry limit.

© Copyright Microsoft Corporation. All rights reserved.

Very high numbers of repositories and tags can impact the performance of your registry. Periodically delete unused repositories, tags, and images as part of your registry maintenance routine. Deleted registry resources like repositories, images, and tags *cannot* be recovered after deletion.

## Build and manage containers with tasks

ACR Tasks provides cloud-based container image building for platforms including Linux, Windows, and ARM. It can automate OS and framework patching for your Docker containers.

### Task scenarios

ACR Tasks supports several scenarios to build and maintain container images and other artifacts:

- Quick task
- Automatically triggered tasks
- Multi-step task

© Copyright Microsoft Corporation. All rights reserved.

### Quick task

Build and push a single container image to a container registry on-demand, in Azure, without needing a local Docker Engine installation.

Before you commit your first line of code, ACR Tasks's quick task feature can provide an integrated development experience by offloading your container image builds to Azure. With quick tasks, you can verify your automated build definitions and catch potential problems prior to committing your code.

### Trigger task on source code update

Trigger a container image build or multi-step task when code is committed, or a pull request is made or updated, to a public or private Git repository in GitHub or Azure DevOps. For example, configure a build task with the Azure CLI command `az acr task create` by specifying a Git repository and optionally a branch and Dockerfile. When your team updates code in the repository, an ACR Tasks-created webhook triggers a build of the container image defined in the repo.

### Trigger on base image update

You can set up an ACR task to track a dependency on a base image when it builds an application image. When the updated base image is pushed to your registry, or a base image is updated in a public repo such as in Docker Hub, ACR Tasks can automatically

build any application images based on it.

### **Schedule a task**

Optionally schedule a task by setting up one or more timer triggers when you create or update the task. Scheduling a task is useful for running container workloads on a defined schedule, or running maintenance operations or tests on images pushed regularly to your registry.

### **Multi-step tasks**

Extend the single image build-and-push capability of ACR Tasks with multi-step, multi-container-based workflows.

Multi-step tasks, defined in a YAML file specify individual build and push operations for container images or other artifacts. They can also define the execution of one or more containers, with each step using the container as its execution environment. For example, you can create a multi-step task that automates the following:

- Build a web application image
- Run the web application container
- Build a web application test image
- Run the web application test container, which performs tests against the running application container
- If the tests pass, build a Helm chart archive package
- Perform a helm upgrade using the new Helm chart archive package

## Dockerfile

- If you want to create a custom container you will need to understand the elements of a Dockerfile.
- A Dockerfile is a text file that contains the instructions we use to build and run a Docker image.
- The following aspects of the image are defined:
  - The base or parent image we use to create the new image
  - Commands to update the base OS and install additional software
  - Build artifacts to include, such as a developed application
  - Services to expose, such a storage and network configuration
  - Command to run when the container is launched

© Copyright Microsoft Corporation. All rights reserved.

<https://docs.docker.com/reference/dockerfile/>

## Explore elements of a Dockerfile

```
# Use the .NET 6 runtime as a base image
FROM mcr.microsoft.com/dotnet/runtime:6.0

# Set the working directory to /app
WORKDIR /app

# Copy the contents of the published app to the container's /app directory
COPY bin/Release/net6.0/publish/ .

# Expose port 80 to the outside world
EXPOSE 80

# Set the command to run when the container starts
CMD ["dotnet", "MyApp.dll"]
```

© Copyright Microsoft Corporation. All rights reserved.

**FROM mcr.microsoft.com/dotnet/runtime:6.0:** This command sets the base image to the .NET 6 runtime, which is needed to run .NET 6 apps.

**WORKDIR /app:** Sets the working directory to /app, which is where app files are copied.

**COPY bin/Release/net6.0/publish/ .:** Copies the contents of the published app to the container's /app directory. We assume that the .NET 6 app is built and published to the bin/Release/net6.0/publish directory.

**EXPOSE 80:** Exposes port 80, which is the default HTTP port, to the outside world. Change this line accordingly if your app listens on a different port.

**CMD ["dotnet", "MyApp.dll"]:** The command to run when the container starts. In this case, we're running the dotnet command with the name of our app's DLL file (MyApp.dll). Change this line to match your app's name and entry point.

## Explore elements of a Dockerfile ( 1 of 2)

# Step 1: Specify the parent image for the new image  
FROM ubuntu:18.04

# Step 2: Update OS packages and install additional software  
RUN apt -y update && apt install -y wget nginx software-properties-common apt-transport-https \  
 && wget -q <URL>/ubuntu/18.04/packages-microsoft-prod.deb -O packages-microsoft-prod.deb \  
 && dpkg -i packages-microsoft-prod.deb \  
 && add-apt-repository universe \  
 && apt -y update \  
 && apt install -y dotnet-sdk-3.0

# Step 3: Configure Nginx environment  
CMD service nginx start

# Step 4: Configure Nginx environment  
COPY ./default /etc/nginx/sites-available/default

© Copyright Microsoft Corporation. All rights reserved.



## Explore elements of a Dockerfile ( 2 of 2 )

```
# STEP 5: Configure work directory  
WORKDIR /app
```

```
# STEP 6: Copy website code to container  
COPY ./website/. .
```

```
# STEP 7: Configure network requirements  
EXPOSE 80:8080
```

```
# STEP 8: Define the entry point of the process that runs in the container  
ENTRYPOINT ["dotnet", "website.dll"]
```

© Copyright Microsoft Corporation. All rights reserved.

## Exercise: Build and run a container image by using Azure Container Registry Tasks

In this exercise you learn how to use ACR Tasks to create a registry and build, push, and run an image in the ACR.

### Objectives

- Create an Azure Container Registry
- Build and push image from a Dockerfile
- Verify the results
- Run the image in the ACR
- Clean up resources

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/training/modules/publish-container-image-to-azure-container-registry/6-build-run-image-azure-container-registry>

## Summary and knowledge check

In this module, you learned how to:

- Explain the features and benefits Azure Container Registry offers
- Describe how to use ACR Tasks to automate builds and deployments
- Explain the elements in a Dockerfile
- Build and run an image in the ACR by using Azure CLI

- 1 Which Azure Container Registry option supports geo-replication to manage a single registry across multiple regions?
- 2 Which Azure container registry tiers benefit from encryption-at-rest?

© Copyright Microsoft Corporation. All rights reserved.

1. The premium tier adds geo-replication as a feature.
2. Encryption-at-rest is supported in all three tiers.

## Module 2: Run container images in Azure Container Instances

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/training/modules/create-run-container-images-azure-container-instances/>

## Learning objectives

- Describe the benefits of Azure Container Instances and how resources are grouped.
- Deploy a container instance in Azure by using the Azure CLI.
- Start and stop containers using policies.
- Set environment variables in your container instances.
- Mount file shares in your container instances.

## Introduction

- Azure Container Instances (ACI) offers the fastest and simplest way to run a container in Azure.
- No requirement to manage any virtual machines and without having to adopt a higher-level service.

© Copyright Microsoft Corporation. All rights reserved.

## Explore Azure Container Instances ( 1 of 3 )

| Feature                             | Description                                                                                                    |
|-------------------------------------|----------------------------------------------------------------------------------------------------------------|
| Fast startup times                  | Containers can start in seconds without the need to provision and manage VMs                                   |
| Public IP connectivity and DNS name | Containers can be directly exposed to the internet with an IP address and a fully qualified domain name (FQDN) |
| Hypervisor-level security           | Container applications are as isolated in a container as they would be in a VM                                 |
| Custom sizes                        | ACI provides optimum utilization by allowing exact specifications of CPU cores and memory                      |
| Persistent storage                  | Containers support direct mounting of Azure Files shares                                                       |
| Linux and Windows containers        | The same API is used to schedule both Linux and Windows containers                                             |
| Co-scheduled groups                 | Container Instances supports scheduling of multicontainer groups that share host machine resources             |
| Virtual network deployment          | Container Instances can be deployed into an Azure virtual network                                              |

© Copyright Microsoft Corporation. All rights reserved.

- **Fast startup:** ACI can start containers in Azure in seconds, without the need to provision and manage VMs
- **Container access:** ACI enables exposing your container groups directly to the internet with an IP address and a fully qualified domain name (FQDN)
- **Hypervisor-level security:** Isolate your application as completely as it would be in a VM
- **Customer data:** The ACI service stores the minimum customer data required to ensure your container groups are running as expected
- **Custom sizes:** ACI provides optimum utilization by allowing exact specifications of CPU cores and memory
- **Persistent storage:** Mount Azure Files shares directly to a container to retrieve and persist state
- **Linux and Windows:** Schedule both Windows and Linux containers using the same API.

For scenarios where you need full container orchestration, including service discovery across multiple containers, automatic scaling, and coordinated application upgrades, we recommend Azure Kubernetes Service (AKS).

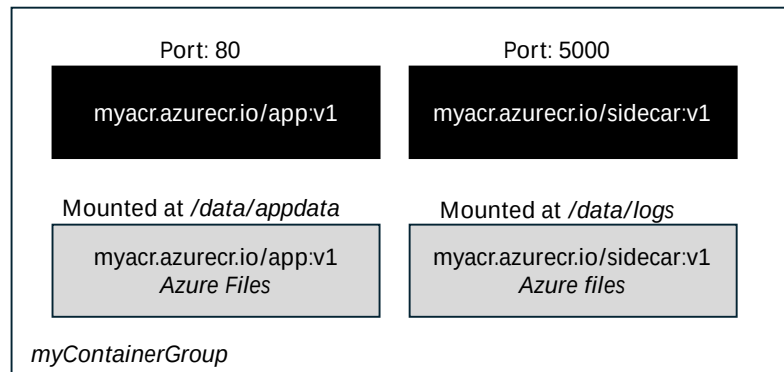
## Explore Azure Container Instances ( 2 of 3 )

### Container groups

The top-level resource in Azure Container Instances is the container group.

The containers in a container group share a lifecycle, resources, local network, and storage volumes.

DNS name label: *myapp.eastus.azurecontainer.io*  
Ports exposed: 80



© Copyright Microsoft Corporation. All rights reserved.

This example container group:

- Is scheduled on a single host machine.
- Is assigned a DNS name label.
- Exposes a single public IP address, with one exposed port.
- Consists of two containers. One container listens on port 80, while the other listens on port 5000.
- Includes two Azure file shares as volume mounts, and each container mounts one of the shares locally.

**Note:** Multi-container groups currently support only Linux containers. For Windows containers, Azure Container Instances only supports deployment of a single instance.



## Explore Azure Container Instances ( 3 of 3 )

### Deployment

- There are two common ways to deploy a multi-container group: ARM template or a YAML file.

### Resource allocation

- ACI allocates resources such as CPUs, memory, and optionally GPUs (preview) to a container group by adding the resource requests of the instances in the group.

### Networking

- Container groups share an IP address and a port namespace on that IP address.

### Storage

- Specify external volumes to mount within a container group.
- Map those volumes into specific paths within the individual containers in a group.

### Common scenarios

- Multi-container groups are useful in cases where you want to divide a single functional task into a small number of container images.

© Copyright Microsoft Corporation. All rights reserved.

### Deployment

- A Resource Manager template is recommended when you need to deploy additional Azure service resources (for example, an Azure Files share) when you deploy the container instances.
- Due to the YAML format's more concise nature, a YAML file is recommended when your deployment includes only container instances.

### Resource allocation

Taking CPU resources as an example, if you create a container group with two instances, each requesting 1 CPU, then the container group is allocated 2 CPUs.

### Networking

To enable external clients to reach a container within the group, you must expose the port on the IP address and from the container. Because containers within the group share a port namespace, port mapping isn't supported. Containers within a group can reach each other via localhost on the ports that they have exposed, even if those ports aren't exposed externally on the group's IP address.

### Storage

Supported volumes include:

- Azure file share
- Secret
- Empty directory
- Cloned git repo

### **Common scenarios**

These images can then be delivered by different teams and have separate resource requirements.

Example usage could include:

- A container serving a web application and a container pulling the latest content from source control.
- An application container and a logging container. The logging container collects the logs and metrics output by the main application and writes them to long-term storage.
- An application container and a monitoring container. The monitoring container periodically makes a request to the application to ensure that it's running and responding correctly, and raises an alert if it's not.
- A front-end container and a back-end container. The front end might serve a web application, with the back end running a service to retrieve data.

## Exercise: Deploy a container instance by using the Azure CLI

In this exercise you learn how to use the Azure CLI in the Azure Cloud Shell to create and run a container in Azure Container Instances.

### Objectives

- Create the resource group
- Create a container
- Verify the container is running
- Clean up resources

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/training/modules/create-run-container-images-azure-container-instances/3-run-azure-container-instances-cloud-shell>

## Run containerized tasks with restart policies ( 1 of 2 )

### Overview

With a configurable restart policy, you can specify that your containers are stopped when their processes have completed.

### Container restart policy

When you create a container group in Azure Container Instances, you can specify one of three restart policy settings:

- Always
- Never
- OnFailure

© Copyright Microsoft Corporation. All rights reserved.

### Container restart policy

**Always** - Containers in the container group are always restarted. This is the default setting applied when no restart policy is specified at container creation.

**Never** - Containers in the container group are never restarted. The containers run at most once.

**OnFailure** - Containers in the container group are restarted only when the process executed in the container fails (when it terminates with a nonzero exit code). The containers are run at least once.

## Set environment variables in container instances ( 1 of 2 )

### YAML example

- Set a secure environment variable by specifying the `secureValue` property instead of the regular value for the variable's type.
- The two variables defined in the YAML demonstrate the two variable types.

```
az container create \  
  --resource-group myResourceGroup \  
  --name mycontainer2 \  
  --image mcr.microsoft.com/azuredocs/aci-wordcount:latest \  
  --restart-policy OnFailure \  
  --environment-variables 'NumWords'='5' 'MinLength'='8'
```

© Copyright Microsoft Corporation. All rights reserved.

You would deploy the YAML example file with the following command:

```
az container create --resource-group myResourceGroup \  
  --file secure-env.yaml
```

## Set environment variables in container instances ( 2 of 2 )

- Provides dynamic configuration of the application or script run by the container.
- ACI supports both Windows and Linux containers to pass secrets as environment variables
- In the example two variables are passed to the container when it is created.

```
az container create \  
  --resource-group myResourceGroup \  
  --name mycontainer2 \  
  --image mcr.microsoft.com/azuredocs/aci-wordcount:latest \  
  --restart-policy OnFailure \  
  --environment-variables 'NumWords'='5' 'MinLength'='8'
```

## Mount an Azure file share in Azure Container Instances ( 1 of 2 )

### Overview

- By default, Azure Container Instances are stateless. If the container crashes or stops, all of its state is lost.
- To persist state beyond the lifetime of the container, you must mount a volume from an external store.

### Limitations

- You can only mount Azure Files shares to Linux containers.
- Azure file share volume mount requires the Linux container run as root.
- Azure File share volume mounts are limited to CIFS support.

## Mount an Azure file share in Azure Container Instances ( 2 of 2 )

### Deploy container and mount volume - YAML

- You can also deploy a container group and mount a volume in a container with the Azure CLI and a YAML template.

### Mount multiple volumes

- To mount multiple volumes in a container instance, you must deploy using an Azure Resource Manager template or a YAML file.
- To use a template or YAML file, provide the share details and define the volumes by populating the volumes array in the properties section of the template.

© Copyright Microsoft Corporation. All rights reserved.

Deploying by YAML template is the preferred method when deploying container groups consisting of multiple containers.



## Summary and knowledge check

In this module, you learned how to:

- Describe the benefits of Azure Container Instances and how resources are grouped
- Deploy a container instance in Azure by using the Azure CLI
- Start and stop containers using policies
- Set environment variables in your container instances
- Mount file shares in your container instances

- 1 What method is recommended when deploying a multi-container group that includes only containers?
- 2 What is the purpose of a restart policy in Azure Container Instances?

© Copyright Microsoft Corporation. All rights reserved.

1. Due to the YAML format's more concise nature, a YAML file is recommended when your deployment includes only container instances.
2. The restart policy allows you to specify when and how containers should be restarted, based on the desired behavior. This can help optimize resource usage and ensure that tasks are completed successfully.

## Module 3: Implement Azure Container Apps

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/training/modules/implement-azure-container-apps/>

## Learning objectives

- Describe the features benefits of Azure Container Apps
- Deploy container app in Azure by using the Azure CLI
- Utilize Azure Container Apps built-in authentication and authorization
- Create revisions and implement app secrets

## Introduction

Azure Container Apps provides the flexibility you need with a serverless container service built for microservice applications and robust autoscaling capabilities without the overhead of managing complex infrastructure.

© Copyright Microsoft Corporation. All rights reserved.

## Explore Azure Container Apps

Azure Container Apps enables you to run microservices and containerized applications on a serverless platform that runs on top of Azure Kubernetes Service.

- Supports dynamic scaling based on any KEDA-supported scaler
- Container apps are deployed to a single Container Apps environment, which acts as a secure boundary around groups of container apps.
- Independently develop, upgrade, version, and scale core areas of functionality in an overall system.
- Native Distributed Application Runtime (Dapr) integration

© Copyright Microsoft Corporation. All rights reserved.

## Exercise: Deploy a container app

In this exercise you create a secure Container Apps environment and deploy container app.

### Objectives

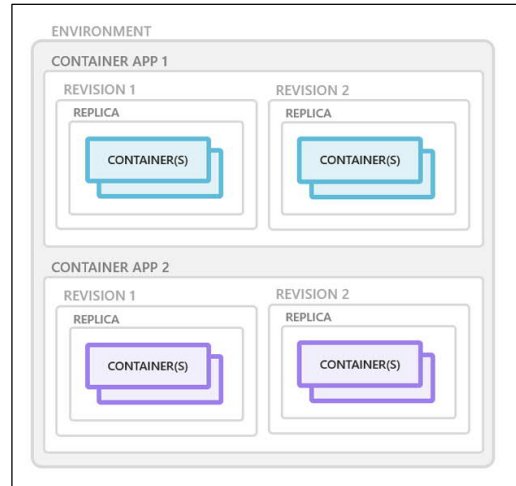
- Prepare your environment
- Create an Azure Container Apps environment
- Create a container app
- Verify deployment
- Clean up resources

© Copyright Microsoft Corporation. All rights reserved.

<https://learn.microsoft.com/training/modules/implement-azure-container-apps/3-exercise-deploy-app>

## Explore containers in Azure Container Apps

- Containers for an Azure Container App are grouped together in pods inside revision snapshots.
- Can define multiple containers in a single container app to implement the sidecar pattern.
- Deploy images hosted on private registries by providing credentials in the Container Apps configuration.



© Copyright Microsoft Corporation. All rights reserved.

Azure Container Apps manages the details of Kubernetes and container orchestration for you. Containers in Azure Container Apps can use any runtime, programming language, or development stack of your choice.

## Implement authentication and authorization in Azure Container Apps

- Container Apps uses federated identity, in which a third-party identity provider manages the user identities and authentication flow for you.
  - Available identity providers include: Microsoft Identity Platform, Facebook, GitHub, Google, Twitter, and any OpenID Connect provider.
- The authentication flow is the same for all providers, but differs depending on whether you want to sign in with the provider's SDK.
  - Without provider SDK (server-directed flow or server flow): The application delegates federated sign-in to Container Apps. Delegation is typically the case with browser apps, which presents the provider's sign-in page to the user.
  - With provider SDK (client-directed flow or client flow): The application signs users in to the provider manually and then submits the authentication token to Container Apps for validation.

© Copyright Microsoft Corporation. All rights reserved.



# Manage revisions and secrets in Azure Container Apps

## Revisions

- Azure Container Apps implements container app versioning by creating revisions.
- Control which revisions are active, and the external traffic that is routed to each active revision.
- The `az containerapp update` command can modify environment variables, compute resources, scale parameters, and deploy a different image.
- If the update includes revision-scope changes, a new revision is generated.

## Secrets

- Secrets are defined at the application level, secured values are available to container apps.
- Each application revision can reference one or more secrets.
- When you create a container app, secrets are defined using the `--secrets` parameter.

© Copyright Microsoft Corporation. All rights reserved.

## Explore Dapr integration with Azure Container Apps

The Distributed Application Runtime (Dapr) provides capabilities for enabling application intercommunication, whether through messaging via pub/sub or reliable and secure service-to-service calls.

| Dapr API                      | Description                                                                                                                |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Service-to-service invocation | Discover services and perform reliable, direct service-to-service calls with automatic mTLS authentication and encryption. |
| State management              | Provides state management capabilities for transactions and CRUD operations.                                               |
| Pub/sub                       | Allows publisher and subscriber container apps to intercommunicate via an intermediary message broker.                     |
| Bindings                      | Trigger your applications based on events.                                                                                 |
| Actors                        | Dapr actors are message-driven, single-threaded, units of work designed to quickly scale.                                  |
| Observability                 | Send tracing information to an Application Insights backend.                                                               |
| Secrets                       | Access secrets from your application code or reference secure values in your Dapr components.                              |
| Configuration                 | Access app configuration and be notified of updates.                                                                       |

© Copyright Microsoft Corporation. All rights reserved.

## Summary and knowledge check

In this module, you learned how to:

- Describe the features benefits of Azure Container Apps
- Deploy container app in Azure by using the Azure CLI
- Utilize Azure Container Apps built-in authentication and authorization
- Create revisions and implement app secrets

1

What is a revision in Azure Container Apps?

© Copyright Microsoft Corporation. All rights reserved.

1. A revision is an immutable snapshot of a container app version.

## Discussion and lab

© Copyright Microsoft Corporation. All rights reserved.



## Group discussion questions

- What factors should you consider when deciding between Azure Container Instances and Azure Container Apps as a deployment target?
- Describe the architecture of an Azure Container App environment. How could some of your apps work within that architecture?
- Describe at least two elements of a Dockerfile. What tools would you use to create a container image?

© Copyright Microsoft Corporation. All rights reserved.

## Lab 05: Deploy compute workloads by using images and containers

In this lab, you will explore how to create and deploy containers to the Azure Container Registry using a .NET application and docker files. And also deploy a containerized solution to Azure Container Apps.

<http://aka.ms/az204labs>

- Exercise 1: Create a Docker container image and deploy it to Azure Container Registry
- Exercise 2: Deploy an Azure container instance
- Exercise 3: Create a secure Container Apps environment and deploy container app

© Copyright Microsoft Corporation. All rights reserved.

End of presentation

© Copyright Microsoft Corporation. All rights reserved.

